

Building a Modern Personal Agent Harness

Memory • browsing the internet • shopping • travel booking • negotiating late returns

System diagrams, 14 architecture variants, a prototype stack for thousands of users,
and the path to millions.

Memory

Tools

Policy

Durable

Evals

Agenda

1. What a personal agent must do
2. Reference architecture
 - agent loop, memory, tools, browser, safety, payments
3. Use-case flows
 - shopping, travel, negotiating a late return
4. 14 architecture variants + comparison
5. Prototype: 0 → thousands of users
 - full technical stack, code sketch, 12-week plan, cost
6. Scaling to millions of users
 - capacity math, cell architecture, cost-down, reliability, security, evals
7. Team, risks, takeaways

Vendor and protocol names are examples of each category as of Sept 2026; costs are order-of-magnitude estimates at list prices – validate before committing.

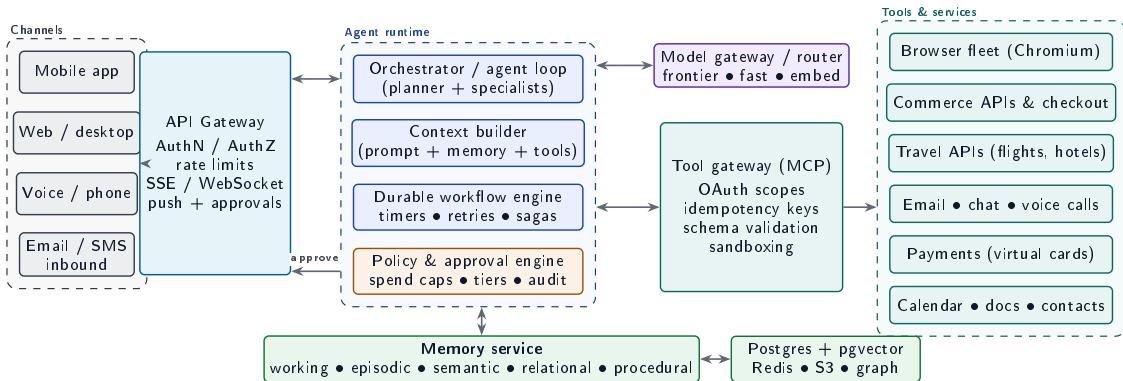
What a personal agent must do

Job	Example request	Hard parts	Capabilities needed
Research & browse	"Best stroller for city + jogging, under \$600"	untrusted web content, freshness, SEO spam	search, fetch, browser, citations, preference memory
Shopping	"Reorder my usual coffee from the cheapest reliable seller"	checkout flows, stock, fraud, payment	commerce APIs, browser checkout, virtual cards, order tracking
Travel	"NYC→Lisbon Oct 12–19, aisle, under \$900, use miles if better"	volatile fares, holds, loyalty, re-booking	flight/hotel APIs, calendar, price watch, durable timers
Negotiation & service	"Return these boots – it's day 35 of a 30-day window"	persuasion within ethics, multi-channel, days of waiting	email, chat, voice, policy reasoning, follow-ups
Life admin	"Cancel the gym, dispute this charge, renew my passport"	identity, forms, 2FA, legal nuance	form filling, document vault, human hand-off

Design principles

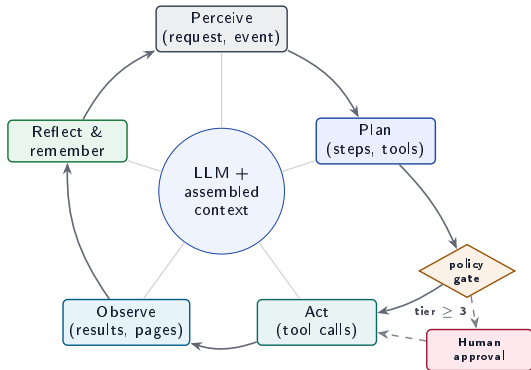
1. **Durable by default** – a return dispute lives for weeks; every step is checkpointed and resumable.
2. **Memory is the product** – sizes, seats, loyalty numbers, merchants you trust, what worked last time.
3. **API first, browser second, human last** – use the most reliable channel that covers the task.
4. **Least privilege + explicit consent** – scoped tokens, per-task spend caps, approval on anything irreversible.
5. **Untrusted content is data, never instructions** – web pages, emails and merchant chats cannot steer the agent.
6. **Observable & replayable** – traces, screenshots, audit log for every action.
7. **Cost-aware** – model routing, prompt caching, compile repeated flows into scripts.
8. **Honest agent** – discloses it is an AI acting for a user; never misstates facts while negotiating.

Reference architecture



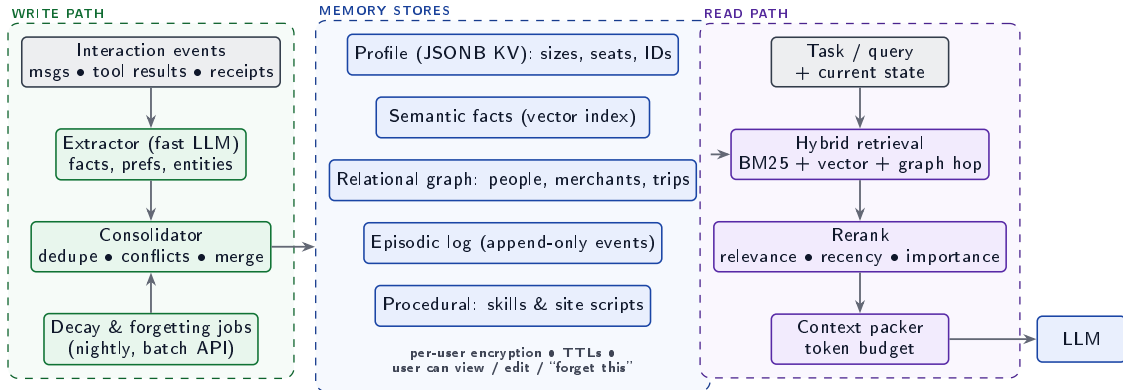
Cross-cutting: OpenTelemetry traces • LLM traces & evals • immutable audit log • secrets vault / KMS • per-user encryption • feature flags

The agent loop



- **Context assembly** in cache-friendly order: frozen system prompt → tool schemas → user profile → retrieved memories → task state.
- **Parallel tool calls** when steps are independent (search 6 merchants at once).
- **Verifier pass** before side effects: “does this cart match the request, size and budget?”
- **Stop conditions:** goal verified, budget (tokens / \$ / wall-clock), max steps, or needs-human.
- **Every step is an event** in a durable log ⇒ crash-safe and resumable after days of waiting.

Memory architecture: write path and read path

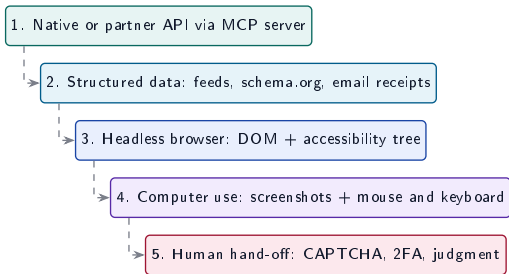


Memory types

Type	Holds	Example	Store	Written / read
Working	current task scratchpad	“cart has 2 items, awaiting approval”	context window + Redis	every step
Episodic	what happened, when	“Sep 3: merchant agreed to extend return by 10 days”	append-only event log (Postgres → S3)	after each step / similar tasks, audits
Semantic	durable facts & preferences	“shoe 10.5 US wide; aisle seat; vegetarian”	pgvector + JSONB profile	post-conversation extraction / every task
Relational	entities and links	“Delta SkyMiles #; partner’s birthday; merchant X policy”	graph tables (or Neo4j later)	extraction / multi-hop questions
Procedural	how to get things done	“checkout script for merchant X; negotiation playbook v3”	versioned skill library (code + prompts)	after successful runs / planning

- Memory writes are **proposals**: provenance (which message / page), confidence, and expiry are stored with every fact.
- Facts extracted from untrusted content (web, merchant emails) are tagged and never become instructions.

Tool layer: API first, browser second, human last

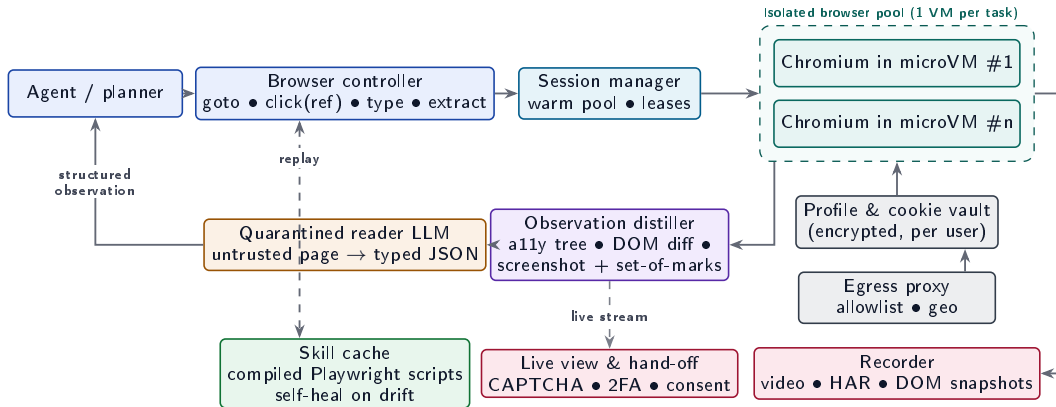


- Every capability is exposed as a **typed tool** (MCP) with scopes, rate limits and idempotency.
- The planner picks the **highest rung** that covers the site; it falls down a rung on failure.
- Successful browser runs are **compiled into scripts** (procedural memory) so rung 3 behaves like rung 1 next time.
- Human hand-off is a first-class tool: live browser view streamed to the user's phone.

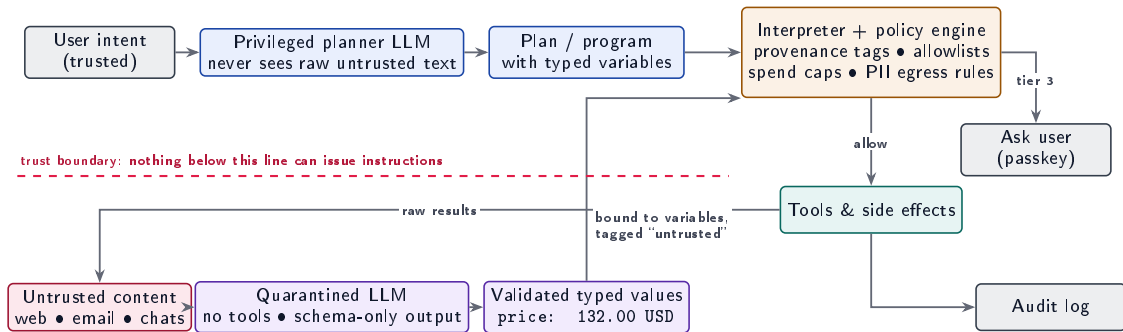
Tool catalog (example vendors per category)

Category	Building blocks
Search & fetch	Claude web search / web fetch server tools; Brave, Exa or Tavily search APIs; readability extraction
Browser	Playwright over CDP; hosted browsers (Browserbase, Steel, Anchor) or self-hosted Chromium in Firecracker / gVisor sandboxes; computer-use tool for pixel-level control
Commerce	Shopify Storefront / catalog APIs, retailer & affiliate product APIs, Agentic Commerce Protocol (ACP) merchants, browser checkout as fallback, order-tracking via email parsing
Travel	Duffel (flights incl. NDC content), GDS (Amadeus, Sabre), hotel APIs (Expedia Rapid, Booking.com), flight-status feeds for delays & rebooking
Communication	Gmail API / Microsoft Graph (mail), Twilio (SMS + voice / SIP), realtime speech-to-speech models for phone calls, merchant live chat via browser
Payments	Stripe Issuing virtual cards with real-time authorization; network agent tokens (Visa Intelligent Commerce, Mastercard Agent Pay); AP2 payment mandates as they mature
Personal data	Google / Microsoft calendar & contacts, Plaid (transactions for price-drop & subscription detection), encrypted document vault (passport, receipts)
Protocols	MCP (agent ↔ tools), A2A (agent ↔ agent), ACP / AP2 (checkout & payment authorization)

Browser agent subsystem



Safety: containing prompt injection (dual-LLM + policy engine)



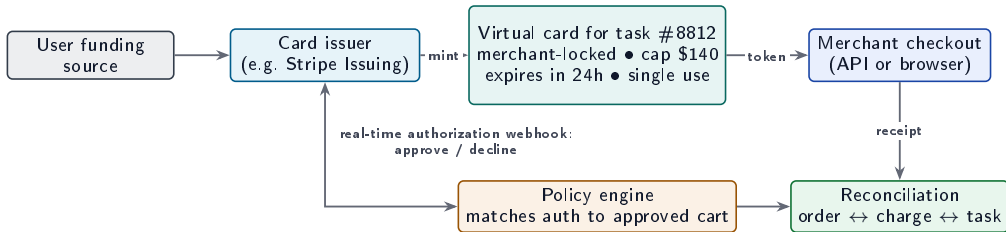
Permission tiers and injection defenses

Tier	Actions	Policy
0	read-only: search, browse, read email, check prices	automatic
1	reversible: add to cart, draft email, hold a fare	auto + notify
2	small commitments within user limits (e.g. $\leq \$50$ at trusted merchant)	auto if policy allows
3	money, irreversible, identity: checkout, ticketing, cancellations, calling on your behalf, sharing PII	explicit approval, step-up auth
4	credential entry on unknown domains, deceptive claims, illegal requests	always deny

Defense in depth

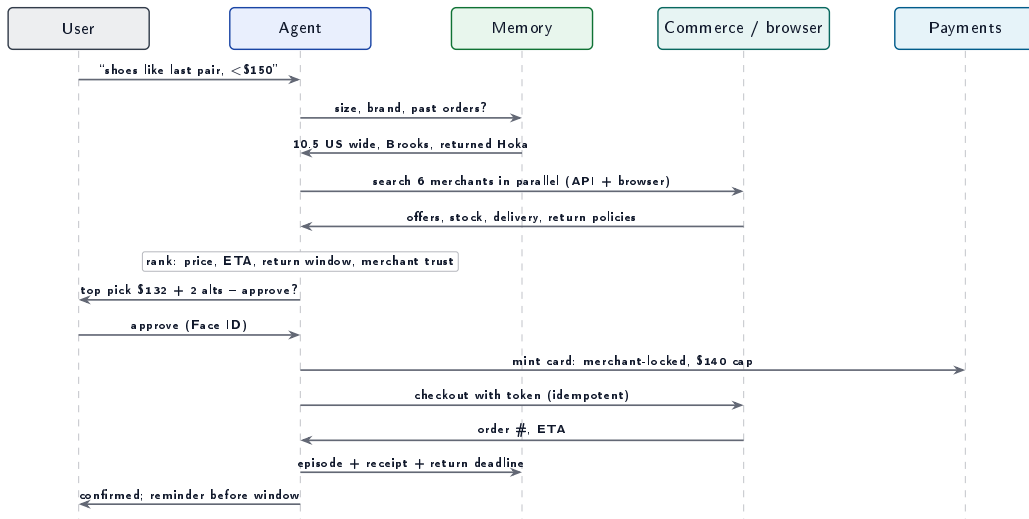
- Separate **instructions** (user) from **data** (world); quarantine readers.
- **Provenance tags** flow with every value; policy checks data flow, not just the call.
- Domain & merchant **allowlists**; egress proxy blocks exfiltration.
- **Virtual cards** capped per task $\Rightarrow$ worst case is bounded.
- Injection red-team suite runs in CI on every prompt / model change.

Money movement: per-task virtual cards

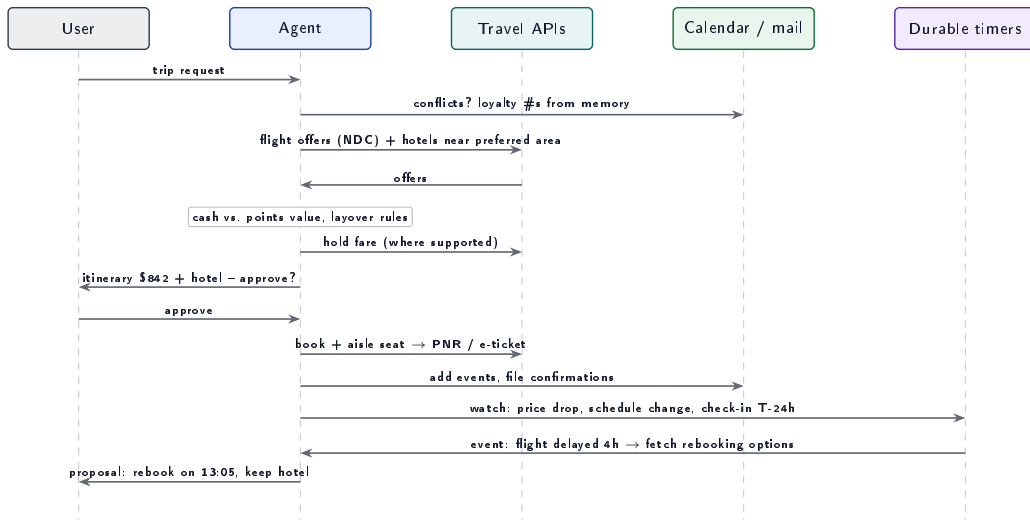


- The agent never sees the user's real card; PAN stays with the issuer $\Rightarrow$ minimal PCI scope.
- Authorization is checked against the *approved* card in real time – a prompt-injected checkout for a different amount or merchant is declined.
- Where merchants support agentic protocols (ACP, AP2, network agent tokens) use signed mandates instead of card-not-present flows.

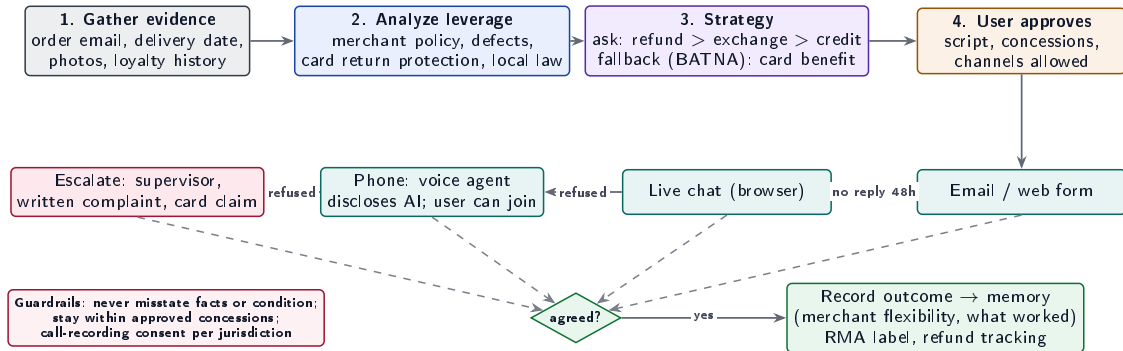
Flow 1 – Shopping: “Running shoes like my last pair, under \$150”



Flow 2 – Travel: “Lisbon Oct 12–19, aisle, under \$900, miles if better”



Flow 3 – Negotiating a late return (day 35 of a 30-day window)



Negotiation playbook (what the agent actually says)

Principles

- Lead with facts and loyalty: “customer since 2019, 14 orders, first return request.”
- Cite the policy precisely, then ask for a specific, easy yes (“a one-time 10-day exception”).
- Offer a graceful fallback (exchange / store credit) only after the primary ask.
- Mention card return protection or chargeback rights only as the user-approved last step.
- Persistent, polite, time-boxed: durable workflow schedules follow-ups.

Sample opening (email, drafted by agent, approved by user)

Hi – I’m an AI assistant writing on behalf of Dana Lee (order #48213). The boots arrived Aug 18 and, unworn, are 5 days past your 30-day window because Dana was travelling. Dana has been a customer since 2019 with no prior returns. Could you make a one-time exception and issue a return label? An exchange for size 10 would also work. Photos of the unworn boots with tags are attached.

Evaluation: success rate, concessions given, turns-to-resolution, and a “would the user sign this?” judge.

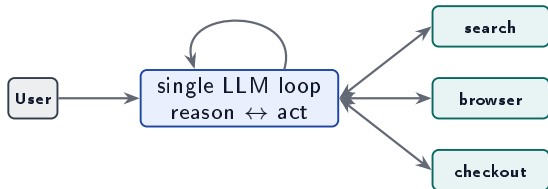
14 architecture variants at a glance

1. Monolithic ReAct agent
2. Planner – executor – verifier
3. Orchestrator + specialist sub-agents
4. Durable workflow graph (LLM at nodes)
5. Event-driven ambient agent
6. Computer-use / browser-native agent
7. API-first MCP tool mesh
8. Local-first hybrid (on-device + cloud)
9. Dual-LLM security architecture
10. Agent-to-agent commerce (A2A / ACP)
11. Voice-first agent
12. Human-in-the-loop concierge
13. Self-improving skill library
14. Memory-centric “digital twin”

They are not mutually exclusive: slide “Recommended composite” shows how to combine them.

Variants 1-2

V1 • Monolithic ReAct agent

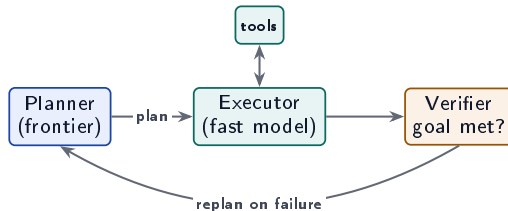


How: one model, one context window, all tools (tool search when there are many).

Best for: week-1 prototype, demos, internal tools.

Watch out: context bloat, weak on multi-day tasks, hard to govern and evaluate.

V2 • Planner – executor – verifier



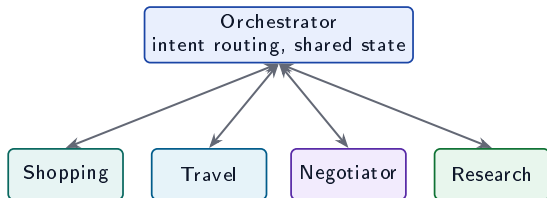
How: explicit step plan; a cheaper model executes; a verifier checks outcome before commit.

Best for: multi-step checkout and booking with verifiable goals.

Watch out: replanning latency; rigid plans on dynamic sites.

Variants 3–4

V3 • Orchestrator + specialist sub-agents

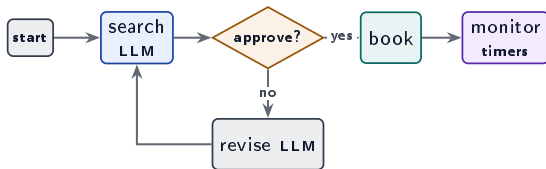


How: a router delegates to domain agents with their own prompts, tools and eval suites.

Best for: growing teams, per-vertical ownership, parallel sub-tasks.

Watch out: hand-off context loss; paying for several contexts per task.

V4 • Durable workflow graph



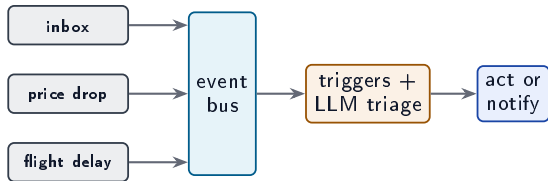
How: deterministic state machine (Temporal / Restate) with LLM calls only at decision nodes; sagas for rollback.

Best for: travel, returns – anything with money and waiting.

Watch out: less flexible for novel, open-ended requests.

Variants 5–6

V5 • Event-driven ambient agent

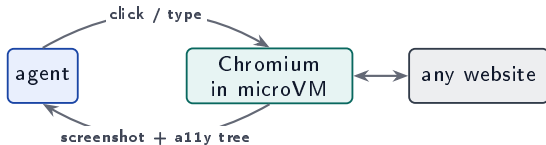


How: agents wake on events (receipts, price drops, flight status, deadlines) and act or notify proactively.

Best for: “chief of staff”: price-adjustment refunds, check-ins, return deadlines.

Watch out: notification fatigue; needs strict per-trigger budgets.

V6 • Computer-use / browser-native



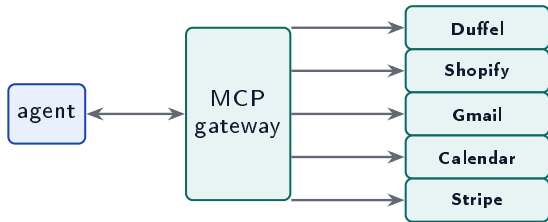
How: the model operates a real browser from screenshots and DOM; zero integrations required.

Best for: the long tail of sites with no API.

Watch out: slowest and most expensive per task; bot defenses and ToS.

Variants 7–8

V7 • API-first MCP tool mesh

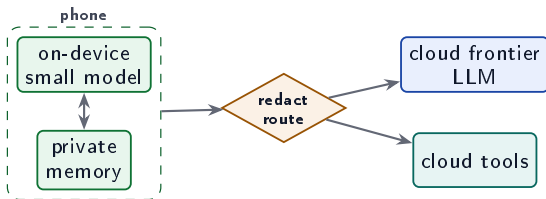


How: every capability is an MCP server with typed schemas and OAuth scopes; browser only as fallback.

Best for: reliability, speed, auditability, low cost.

Watch out: coverage limited to partners; integration effort per vendor.

V8 • Local-first hybrid



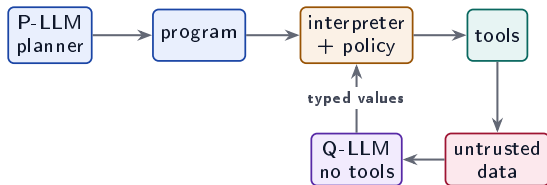
How: on-device model handles intent, PII and memory; the cloud model receives redacted context.

Best for: privacy-sensitive users, offline use, low latency.

Watch out: device fragmentation, sync, weaker local reasoning.

Variants 9–10

V9 • Dual-LLM security architecture

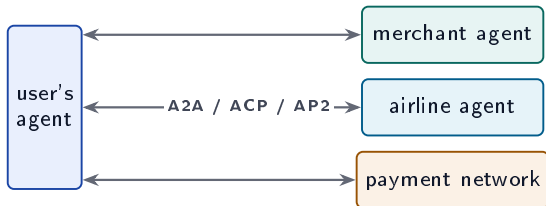


How: the privileged LLM plans without ever reading untrusted text; a quarantined LLM parses pages into typed values; policy enforces data flow.

Best for: payments, PII, high-assurance actions.

Watch out: engineering complexity; some tasks must reason over untrusted text.

V10 • Agent-to-agent commerce



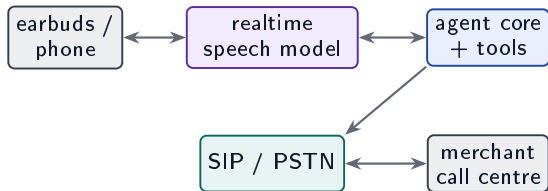
How: your agent negotiates and checks out with merchant and airline agents via open protocols and signed payment mandates.

Best for: future-proof commerce; no scraping; structured offers.

Watch out: early ecosystem; adversarial counter-agents.

Variants 11–12

V11 • Voice-first agent

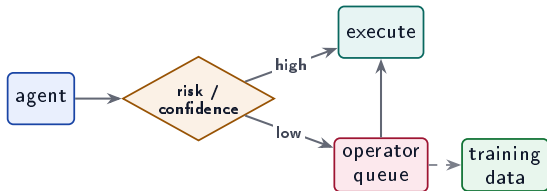


How: realtime speech-to-speech front end; outbound calls handle customer service and reservations.

Best for: negotiation, reservations, hands-free use.

Watch out: AI-call disclosure & consent law (e.g. TCPA), latency, recording rules.

V12 • Human-in-the-loop concierge



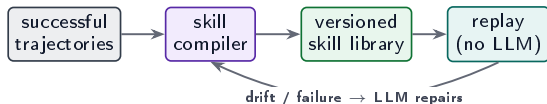
How: a risk/confidence router sends hard cases to human operators who act through the same tools.

Best for: launch quality at small scale; building trust; labeled data.

Watch out: operations cost; the human share must shrink over time.

Variants 13–14

V13 • Self-improving skill library

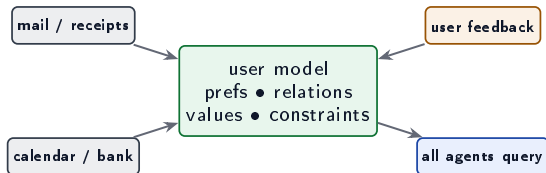


How: trajectories become parameterized scripts, replayed deterministically; the LLM is used only when a site changes.

Best for: high-frequency flows at scale – 5–20× cheaper and faster.

Watch out: site drift; skill verification and versioning.

V14 • Memory-centric “digital twin”



How: a rich, user-editable model of preferences and constraints that every agent consults.

Best for: deep personalization – the long-term moat.

Watch out: privacy, staleness, conflicting preferences.

Comparison matrix

Variant	Ease of build	Reliability	Cost efficiency	Speed	Safety / privacy	Coverage
V1 Monolithic ReAct	●●●	●●●	●●●	●●●	●●●	●●●
V2 Planner-executor-verifier	●●●	●●●	●●●	●●●	●●●	●●●
V3 Orchestrator + specialists	●●●	●●●	●●●	●●●	●●●	●●●
V4 Durable workflow graph	●●●	●●●	●●●	●●●	●●●	●●●
V5 Event-driven ambient	●●●	●●●	●●●	●●●	●●●	●●●
V6 Computer-use browser	●●●	●●●	●●●	●●●	●●●	●●●
V7 API-first MCP mesh	●●●	●●●	●●●	●●●	●●●	●●●
V8 Local-first hybrid	●●●	●●●	●●●	●●●	●●●	●●●
V9 Dual-LLM security	●●●	●●●	●●●	●●●	●●●	●●●
V10 Agent-to-agent	●●●	●●●	●●●	●●●	●●●	●●●
V11 Voice-first	●●●	●●●	●●●	●●●	●●●	●●●
V12 Human-in-the-loop	●●●	●●●	●●●	●●●	●●●	●●●
V13 Skill library	●●●	●●●	●●●	●●●	●●●	●●●
V14 Digital twin	●●●	●●●	●●●	●●●	●●●	●●●

More filled dots = better. Qualitative, relative judgments for a consumer personal-agent product.

Recommended composite

Experience	Chat app + push approvals • proactive nudges (V5) • voice for calls (V11)
Orchestration	Orchestrator + specialists (V3) running on durable workflows (V4)
Reasoning safety	Planner / verifier (V2) • dual-LLM containment + policy engine (V9)
Capabilities	MCP-first tools (V7) → skill replay (V13) → browser / computer use (V6) → A2A (V10) → human (V12)
Knowledge	Memory service → digital twin (V14) • on-device private vault (V8, later)

Prototype V2 V3 V4 V6 V7 V12 + V9-lite Scale add V5 V11 V13 V14 Later V8 V10 as ecosystems mature

Prototype goals (0 → thousands of users)

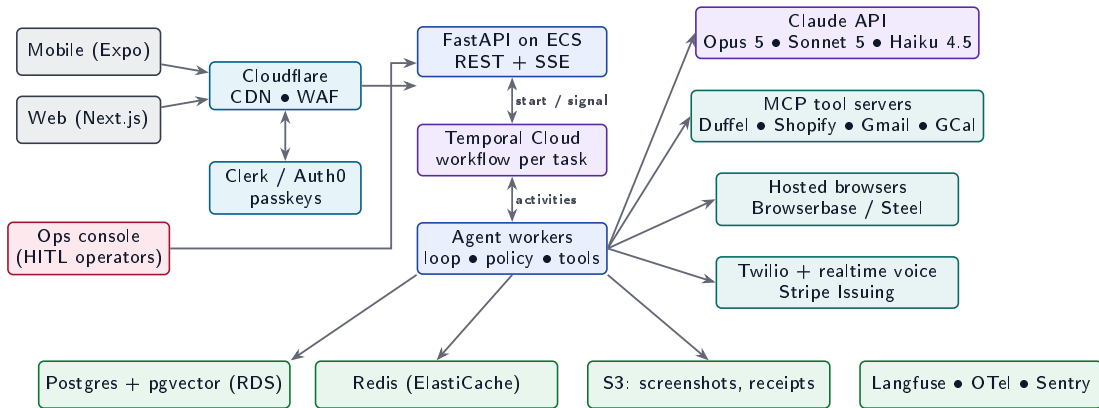
Scope

- 1,000–5,000 invited users, US only, one cloud region.
- Three verticals: shopping, travel, returns & customer service.
- Mobile app + web; approvals via push notification + Face ID / passkey.
- Human-in-the-loop operators review risky or low-confidence steps.

Targets

- Task success $\geq 80\%$ (with HITL), $\geq 65\%$ fully autonomous.
- Zero unauthorized spend (hard caps via virtual cards).
- First response < 2 s; simple tasks done < 3 min.
- 4–6 engineers, 12 weeks, mostly managed services.
- Every task fully traced and replayable.

Prototype architecture (single region, managed services)



Prototype stack in detail (1/2)

Layer	Choice	Why	Alternatives
Clients	Expo / React Native app + Next.js web; SSE streaming; Expo push notifications; biometric approval via passkeys	one TypeScript codebase; approvals where the user is	Swift / Kotlin native
API	Python 3.12, FastAPI + Pydantic v2, uvicorn; REST + SSE; OpenAPI-generated TS client	richest LLM / agent ecosystem	TypeScript (Hono)
Agent runtime	own loop on the Anthropic SDK (tool runner hooks for approval gates); runs inside Temporal activities; MCP clients for tools	full control over policy, caching, retries	LangGraph, Pydantic AI
Models	Claude Opus 5 for planning & negotiation; Sonnet 5 for most tool steps and browser control; Haiku 4.5 for extraction, classification, memory; prompt caching on; Batch API for nightly memory jobs	quality where it matters, cost everywhere else	multi-provider router later
Durable execution	Temporal Cloud: one workflow per task; signals for approvals; timers for follow-ups; activities for tool calls with retries	days-long tasks survive deploys & crashes	Restate, Inngest, DBOS
Streaming & jobs	Redis pub/sub to stream steps to the UI; Temporal schedules for price-watch & reminders	simple, few moving parts	NATS

Prototype stack in detail (2/2)

Layer	Choice	Why	Alternatives
Memory	Postgres 16 + pgvector (HNSW), JSONB profile, entity / edge tables, row-level security by user; Redis for working memory; S3 for artifacts	one database to operate	Turbopuffer, Neo4j (later)
Browser	Playwright over CDP to hosted browsers with session recording and live view for hand-off; a11y-tree + screenshot observations	no fleet to operate yet	self-hosted Chromium
Integrations	one MCP server per integration (Python FastMCP): Duffel, Shopify, Gmail / Graph, Google Calendar, Twilio, Stripe Issuing	typed, scoped, reusable	direct SDK calls
Payments	Stripe Issuing virtual cards + real-time authorization webhook into the policy engine; Stripe Billing for subscriptions	bounded worst case; tiny PCI scope	Lithic, Marqeta
Identity & secrets	Clerk / Auth0 with passkeys; OAuth refresh tokens encrypted with per-user data keys (AWS KMS envelope encryption); step-up auth for tier-3 actions	least privilege	WorkOS, Vault
Observability & evals	OpenTelemetry → Grafana; Langfuse for LLM traces & cost; Sentry; pytest eval suites with LLM judges; browser video replays	debug any task end-to-end	LangSmith, Braintrust
Infra & delivery	AWS: ECS Fargate, RDS, ElastiCache, S3; Terraform; GitHub Actions; GrowthBook feature flags; prompt & model versions in git	boring and fast	Fly.io, GCP

Code sketch: one task = one durable workflow

```
@workflow.defn
class PersonalTask:
    def __init__(self): self.approvals: dict[str, bool] = {}
    @workflow.run
    async def run(self, task: TaskSpec) -> Outcome:
        ctx = await act(build_context, task) # profile+memories+tools
        while not ctx.done:
            step = await act(llm_step, ctx) # Claude: next tool calls
            for call in step.tool_calls:
                verdict = await act(policy_check, call, ctx)
                if verdict.deny:
                    ctx.add_denial(call); continue
                if verdict.needs_approval: # push + passkey
                    await act(request_approval, call)
                    await workflow.wait_condition(
                        lambda: call.id in self.approvals,
                        timeout=timedelta(days=2))
                    if not self.approvals[call.id]:
                        ctx.add_denial(call); continue
                result = await act(run_tool, call,
                                    idem_key=f"{task.id}:{call.id}")
                ctx.observe(result)
            if ctx.waiting_on_merchant: # e.g. return email
                await workflow.sleep(timedelta(hours=48))
        await act(write_memories, ctx)
        return ctx.outcome
    @workflow.signal
    def approve(self, call_id: str, ok: bool):
        self.approvals[call_id] = ok
```

- act = Temporal activity with retries & timeouts; LLM and tool calls are activities, so a crash replays from history, not from scratch.
- Approvals are **signals**; waiting two days costs nothing.
- **Idempotency key** per tool call prevents double purchases on retry.
- The loop is the same for every vertical; specialists differ in prompts, tools and policies.

12-week build plan



Prototype capacity and cost (order of magnitude)

Load assumptions (5,000 users)

- 40% weekly active $\times$ 10 tasks / week $\approx$ 3k tasks / day (<1 task/s).
- Per task: ~ 12 LLM calls, $\sim 25k$ input tokens each ($\sim 80\%$ cache reads), ~ 800 output tokens.
- 30% of tasks use a browser for ~ 4 min; 1% place a phone call.

LLM cost per task

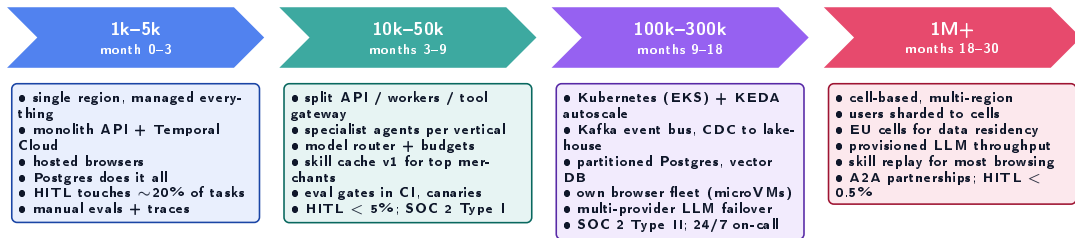
- Sonnet 5 at \$2 / \$10 per M tokens (in / out), cache reads $\sim 0.1\times$: $\approx$ \$0.02 per call $\rightarrow$ \$0.25.
- 2 Opus 5 planning / negotiation calls: $\approx$ \$0.15–0.20.
- $\Rightarrow$ roughly **\$0.40–0.50 per task** before optimization.

Monthly run-rate

Item	\$ / month
LLM tokens ($\sim 90k$ tasks)	35k–45k
Hosted browsers	1k–3k
Voice & telephony	1k–2k
Temporal Cloud	1k–2k
AWS (ECS, RDS, Redis, S3)	3k–6k
Observability, auth, misc SaaS	1k–3k
HITL operators (2–4 FTE)	25k–50k
Total	$\sim 70k$–$110k$

$\approx$ \$15–20 per active user / month $\Rightarrow$ price at \$20–30 / month with usage caps, and start cost-down work before scaling.

Scaling phases: what changes at each order of magnitude



Capacity math for 1 million users

Traffic

- 1M MAU, 35% daily active, 3 tasks / DAU / day $\Rightarrow$ $\sim 1\text{M}$ tasks / day.
- ≈ 12 tasks/s average, ~ 60 /s at peak ($5\times$).

LLM

- 12 calls / task $\Rightarrow \sim 150$ calls/s average, ~ 700 /s peak.
- $\sim 300\text{B}$ input tokens / day (must be $\geq 90\%$ cache reads) and $\sim 10\text{B}$ output tokens / day.
- Needs negotiated / provisioned throughput and multi-provider capacity.

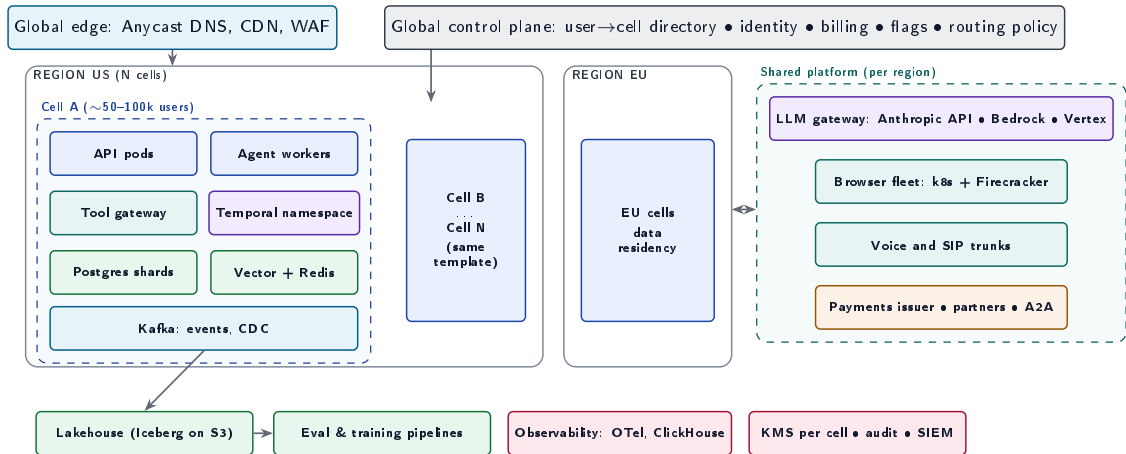
Browsers & voice

- 30% of tasks $\times 4$ min $\Rightarrow \sim 850$ concurrent sessions average, $\sim 4\text{k}$ peak (skill replay cuts this $5\text{--}10\times$).
- 1% of tasks $\times 8$ min calls $\Rightarrow \sim 80\text{k}$ call-minutes / day.

State

- $\sim 1\text{M}$ workflows started / day; $5\text{--}10\text{M}$ long-lived (price watches, return follow-ups).
- Memory: thousands of items / user / year $\Rightarrow$ billions of vectors – always queried per user.

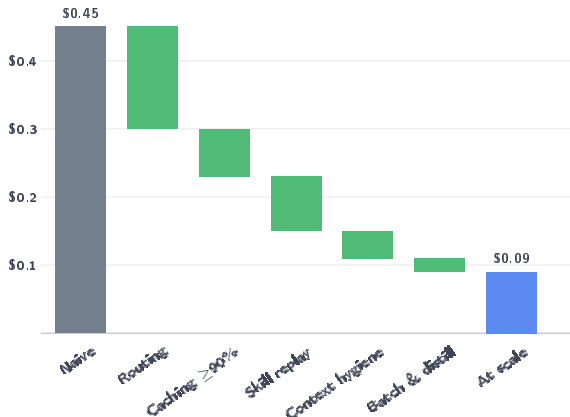
Million-user architecture: cells, regions, shared platform



Cell design principles

- A **cell** is a complete, independent copy of the stack serving a fixed set of users (50–100k). Blast radius of any failure = one cell.
- Users are **pinned to cells** via the global directory; moving a user = replay their data into another cell.
- Cells scale out, not up: new cohort $\Rightarrow$ new cell from the same Terraform / Helm template.
- Rollouts go cell by cell (canary cell first) for code, prompts and model versions.
- **Shared, stateless services** stay regional: LLM gateway, browser fleet, telephony, payments.
- **Temporal**: one namespace per cell; workers autoscaled by KEDA on task-queue backlog.
- **Postgres**: Aurora or Citus, sharded by `user_id`; episodic logs older than 90 days tier to Iceberg.
- **Vector**: per-user namespaces in a multi-tenant vector store (Turbopuffer / Pinecone serverless) or partitioned pgvector – every query is user-scoped, so indexes stay tiny.

LLM cost-down at scale: \$0.45 → ~\$0.09 per task



- **Routing:** ~60% of calls to Haiku 4.5 / Sonnet 5 at low effort; frontier only for planning & negotiation.
- **Caching:** frozen system prompt + tools + profile prefix; append-only transcripts.
- **Skill replay:** repeated merchant flows run as scripts; LLM only on drift.
- **Context hygiene:** clear old tool results, compaction, distilled observations instead of raw DOM.
- **Batch & distill:** memory consolidation via Batch API (50% off); small fine-tuned extractors.
- Per-user token budgets by plan tier; cost per *completed* task is the KPI.

Illustrative waterfall; measure on your own traffic.

Browser, tools and voice at scale

Browser fleet

- Kubernetes + Firecracker (or Kata / gVisor) microVM per session; warm pools per region; ~1–2 GB RAM per session.
- Encrypted per-user browser profiles; secrets injected at the proxy, never shown to the model.
- Compile the top ~500 merchant flows into scripts with self-healing selectors.
- Per-site circuit breakers & rate limits; respect robots / ToS; prefer partner APIs and agentic-commerce protocols.

Tool gateway

- MCP servers behind one gateway: per-user OAuth tokens, scopes, quotas, schema validation, idempotency, response caching.
- Integration SLOs and synthetic probes per vendor; auto-fallback to next rung of the ladder.

Voice

- SIP trunks with regional capacity; realtime voice model streams; recordings stored per consent rules.
- Always disclose the AI; user can barge in from the app at any time.

Reliability and correctness

Side effects

- Idempotency key on every side-effecting call, deduplicated at the tool gateway.
- Sagas with compensations: flight booked, hotel fails $\Rightarrow$ void ticket inside the void window.
- Money: issuer authorization bound to task id; daily reconciliation of orders $\leftrightarrow$ charges $\leftrightarrow$ tasks.

Degradation

- LLM provider failover (Anthropic API $\leftrightarrow$ Bedrock $\leftrightarrow$ Vertex) behind the gateway.
- Non-urgent work queues instead of failing; ambient jobs shed first.

SLOs

API availability	99.95%
First streamed token	< 1.5 s p95
Approval push delivered	< 5 s p95
Task success (per vertical)	$\geq$ 90%
Unauthorized spend	0
Workflow durability	no lost tasks

Game days: provider outage, merchant redesign, region fail-over, Temporal backlog surge, card-issuer outage.

Security, privacy and compliance

Threats

- prompt injection via web & email
- account takeover
- payment fraud
- data exfiltration
- malicious merchants / counter-agents
- misuse of agents at scale

Controls

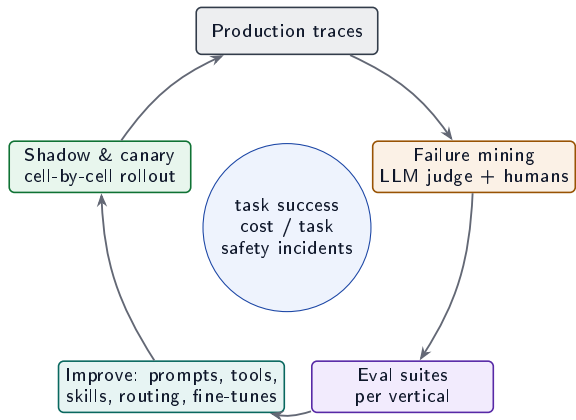
- dual-LLM containment + provenance
- egress allowlists; spend caps
- step-up auth for tier 3
- anomaly detection on agent actions
- red-team suites in CI; bug bounty
- per-user envelope encryption $\Rightarrow$ crypto-shredding on delete

Compliance

- SOC 2 Type II, ISO 27001
- PCI DSS scope minimized via issuer tokens
- GDPR / CCPA: export, delete, EU residency
- EU AI Act transparency duties
- call-recording & AI-voice rules (e.g. TCPA)
- no training on user data without opt-in

User controls: memory viewer / editor, full activity log with replays, per-merchant permissions, global kill switch.

Evals and the quality flywheel



- **Simulation environments:** recorded merchant sites (HAR replay), mock travel APIs, synthetic customer-service agents that negotiate back.
- **Adversarial pages** with injected instructions in every suite.
- Metrics per vertical: success, steps, cost, time, user edits to drafts, approvals denied.
- No prompt, model or skill change ships without passing the eval gate.

Team shape as you scale

	Prototype (1k–5k)	Growth (100k)	Scale (1M+)
Headcount	4–6 engineers + PM / ops	25–40	100+
Agent & product	2 full-stack agent engineers, 1 mobile / web	vertical pods: shopping, travel, service, ambient	+ voice, A2A partnerships, personalization
Platform	1 infra / security generalist	platform, data, browser fleet	SRE per region, cell platform, LLM gateway team
Quality & safety	shared eval duty	evals team, trust & safety	red team, fraud, compliance, privacy engineering
Operations	2–4 HITL operators	HITL + support (<5% of tasks)	support + escalations (<0.5%)
Partnerships	–	merchants, travel suppliers	payment networks, A2A / ACP ecosystems

Risks and mitigations

Risk	Mitigation
Sites block agents / ToS conflicts	API-first, merchant partnerships, agentic-commerce protocols, user-present browsing, respectful rate limits
Prompt injection causes harmful action	dual-LLM containment, provenance-aware policy, approvals, per-task virtual-card caps
Cost per task outgrows revenue	routing, caching, skill replay, budgets per plan tier, cost-per-completed-task KPI
Wrong purchase / booking	verifier pass, explicit approval with summary, sagas & easy undo, liability reserve / insurance
Regulation of AI calls & agents	AI disclosure, consent capture, jurisdiction-aware rules engine, audit trail
User trust & privacy	transparent, editable memory; encryption; no training without opt-in; data residency
Model or provider outage	multi-provider gateway, graceful degradation, queued non-urgent work

Key takeaways

1. A personal agent is a **durable, memory-rich workflow system** with an LLM inside – not a chatbot with tools.
2. Start with the **composite**: orchestrator + specialists on durable workflows, MCP-first tools, browser fallback, policy engine, human-in-the-loop.
3. **Bound the blast radius** before adding autonomy: permission tiers, virtual cards, quarantined readers, audit.
4. The prototype stack is **boring on purpose**: FastAPI, Temporal, Postgres + pgvector, hosted browsers, Claude API, Stripe Issuing.
5. Scaling to millions is mostly **cost-down and blast-radius control**: routing, caching, skill replay, cells, multi-provider capacity.
6. The moat is **memory + evals**: knowing the user and proving the agent gets things right.